```cpp
#pragma once
#include <iostream>
#include <vector>
#include <cmath>
#include "matrix.hpp"

class capaMultiHeadAtencion {
private:
    size_t dimModel_;
    size_t numHeads_;
    size_t dimK_; // dimModel / numHeads

    // Matrices de proyección proyectadas globales y de salida
    Matriz Wq, Wk, Wv, Wo;

    // Acumuladores Adam para Wq, Wk, Wv, Wo
    Matriz mWq, vWq, mWk, vWk, mWv, vWv, mWo, vWo;

    // Caché para Backward Pass
    Matriz X_cache;
    Matriz Q_cache, K_cache, V_cache;
    Matriz H_concat_cache;
    std::vector<Matriz> A_cache_heads;
    std::vector<Matriz> scores_cache_heads;

    double beta1 = 0.9;
    double beta2 = 0.999;
    double epsilon = 1e-8;

public:
    capaMultiHeadAtencion(size_t dimModel, size_t numHeads)
        : dimModel_(dimModel), numHeads_(numHeads), dimK_(dimModel / numHeads)
    {
        Wq = Matriz(dimModel_, dimModel_);
        Wk = Matriz(dimModel_, dimModel_);
        Wv = Matriz(dimModel_, dimModel_);
        Wo = Matriz(dimModel_, dimModel_);

        mWq = Matriz(dimModel_, dimModel_, 0.0); vWq = Matriz(dimModel_, dimModel_, 0.0);
        mWk = Matriz(dimModel_, dimModel_, 0.0); vWk = Matriz(dimModel_, dimModel_, 0.0);
        mWv = Matriz(dimModel_, dimModel_, 0.0); vWv = Matriz(dimModel_, dimModel_, 0.0);
        mWo = Matriz(dimModel_, dimModel_, 0.0); vWo = Matriz(dimModel_, dimModel_, 0.0);

        double limite = std::sqrt(6.0 / (dimModel_ + dimModel_));
        Wq.aleatorizar(-limite, limite);
        Wk.aleatorizar(-limite, limite);
        Wv.aleatorizar(-limite, limite);
        Wo.aleatorizar(-limite, limite);
    }

    Matriz forward(const Matriz& X) {
        X_cache = X;
        size_t seqLen = X.columnas();

        // 1. Proyecciones globales Q, K, V
        Q_cache = Wq.matmul(X); // (dimModel x seqLen)
        K_cache = Wk.matmul(X);
        V_cache = Wv.matmul(X);

        A_cache_heads.resize(numHeads_);
        scores_cache_heads.resize(numHeads_);

        Matriz H_concat(dimModel_, seqLen);
        double escala = std::sqrt(static_cast<double>(dimK_));

        // 2. Procesar cada cabeza independientemente
```

```
    for (size_t h = 0; h < numHeads_; ++h) {
        size_t offsetFila = h * dimK_;

        // Extraer sub-matrices Q_h, K_h, V_h de dimensión (dimK x seqLen)
        Matriz Qh(dimK_, seqLen), Kh(dimK_, seqLen), Vh(dimK_, seqLen);
        for (size_t i = 0; i < dimK_; ++i) {
            for (size_t j = 0; j < seqLen; ++j) {
                Qh(i, j) = Q_cache(offsetFila + i, j);
                Kh(i, j) = K_cache(offsetFila + i, j);
                Vh(i, j) = V_cache(offsetFila + i, j);
            }
        }

        // Scores = (Q_h^T * K_h) / sqrt(dimK)
        Matriz scores = Qh.transpuesta().matmul(Kh);
        for (size_t i = 0; i < seqLen; ++i) {
            for (size_t j = 0; j < seqLen; ++j) {
                scores(i, j) /= escala;
                if (j > i) scores(i, j) = -1e9; // Máscara Causal
            }
        }
        scores_cache_heads[h] = scores;

        // Softmax por fila
        Matriz Ah(seqLen, seqLen);
        for (size_t i = 0; i < seqLen; ++i) {
            Matriz filaTemp(seqLen, 1);
            for (size_t j = 0; j < seqLen; ++j) filaTemp(j, 0) = scores(i, j);
            Matriz filaSoftmax = filaTemp.softmax();
            for (size_t j = 0; j < seqLen; ++j) Ah(i, j) = filaSoftmax(j, 0);
        }
        A_cache_heads[h] = Ah;

        // Head_out = V_h * A_h^T  (dimK x seqLen)
        Matriz headOut = Vh.matmul(Ah.transpuesta());

        // Ensamblar en la matriz concatenada
        for (size_t i = 0; i < dimK_; ++i) {
            for (size_t j = 0; j < seqLen; ++j) {
                H_concat(offsetFila + i, j) = headOut(i, j);
            }
        }
    }

    H_concat_cache = H_concat;

    // 3. Proyección final de salida Wo * H_concat
    return Wo.matmul(H_concat);
}

Matriz backward(const Matriz& gradSalida, double lr, int t) {
    size_t seqLen = gradSalida.columnas();

    // dWo = gradSalida * H_concat^T
    Matriz dWo = gradSalida.matmul(H_concat_cache.transpuesta());
    // dH_concat = Wo^T * gradSalida
    Matriz dH_concat = Wo.transpuesta().matmul(gradSalida);

    actualizarParametro(Wo, dWo, mWo, vWo, lr, t);

    Matriz dQ_total(dimModel_, seqLen, 0.0);
    Matriz dK_total(dimModel_, seqLen, 0.0);
    Matriz dV_total(dimModel_, seqLen, 0.0);

    double escala = std::sqrt(static_cast<double>(dimK_));
```

```
    // Gradientes divididos por cabeza
    for (size_t h = 0; h < numHeads_; ++h) {
        size_t offsetFila = h * dimK_;

        Matriz dHead(dimK_, seqLen);
        Matriz Qh(dimK_, seqLen), Kh(dimK_, seqLen), Vh(dimK_, seqLen);
        for (size_t i = 0; i < dimK_; ++i) {
            for (size_t j = 0; j < seqLen; ++j) {
                dHead(i, j) = dH_concat(offsetFila + i, j);
                Qh(i, j) = Q_cache(offsetFila + i, j);
                Kh(i, j) = K_cache(offsetFila + i, j);
                Vh(i, j) = V_cache(offsetFila + i, j);
            }
        }

        Matriz Ah = A_cache_heads[h];
        Matriz dVh = dHead.matmul(Ah);
        Matriz dAh = dHead.transpuesta().matmul(Vh);

        Matriz dScores(seqLen, seqLen, 0.0);
        for (size_t i = 0; i < seqLen; ++i) {
            for (size_t j = 0; j < seqLen; ++j) {
                double suma = 0.0;
                for (size_t k = 0; k < seqLen; ++k) {
                    double delta = (j == k) ? 1.0 : 0.0;
                    suma += dAh(i, k) * Ah(i, k) * (delta - Ah(i, j));
                }
                dScores(i, j) = suma / escala;
            }
        }

        Matriz dQh = Kh.matmul(dScores.transpuesta());
        Matriz dKh = Qh.matmul(dScores);

        for (size_t i = 0; i < dimK_; ++i) {
            for (size_t j = 0; j < seqLen; ++j) {
                dQ_total(offsetFila + i, j) = dQh(i, j);
                dK_total(offsetFila + i, j) = dKh(i, j);
                dV_total(offsetFila + i, j) = dVh(i, j);
            }
        }
    }

    Matriz dWq = dQ_total.matmul(X_cache.transpuesta());
    Matriz dWk = dK_total.matmul(X_cache.transpuesta());
    Matriz dWv = dV_total.matmul(X_cache.transpuesta());

    actualizarParametro(Wq, dWq, mWq, vWq, lr, t);
    actualizarParametro(Wk, dWk, mWk, vWk, lr, t);
    actualizarParametro(Wv, dWv, mWv, vWv, lr, t);

    return Wq.transpuesta().matmul(dQ_total) +
           Wk.transpuesta().matmul(dK_total) +
           Wv.transpuesta().matmul(dV_total);
}

void guardarBinario(std::ofstream& out) const {
    Wq.guardarBinario(out); Wk.guardarBinario(out);
    Wv.guardarBinario(out); Wo.guardarBinario(out);
}

void cargarBinario(std::ifstream& in) {
    Wq.cargarBinario(in); Wk.cargarBinario(in);
    Wv.cargarBinario(in); Wo.cargarBinario(in);
}
```

```
private:
    void actualizarParametro(Matriz& W, const Matriz& dW, Matriz& mW, Matriz& vW, double
lr, int t) {
        for (size_t i = 0; i < W.filas(); ++i) {
            for (size_t j = 0; j < W.columnas(); ++j) {
                double g = dW(i, j);
                mW(i, j) = beta1 * mW(i, j) + (1.0 - beta1) * g;
                vW(i, j) = beta2 * vW(i, j) + (1.0 - beta2) * (g * g);

                double m_hat = mW(i, j) / (1.0 - std::pow(beta1, t));
                double v_hat = vW(i, j) / (1.0 - std::pow(beta2, t));

                W(i, j) -= (lr * m_hat) / (std::sqrt(v_hat) + epsilon);
            }
        }
    }
};
```